

JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 13 OF 18

Trees, Binary Search Trees, and Tries

Traversal Contracts, Path State, Ordering, Balance, and Prefix Search

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Model recursive tree contracts, own traversal state correctly, exploit BST ordering, and use tries for prefix-shaped search.

DFS | BFS | BST | LCA | BALANCE | TRIES | SERIALIZATION

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to **12 – Binary Search**. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Tree interviews test recursion, state ownership, structural invariants, and the ability to translate ordering into pruning.

Readiness map

Decision	Evidence
START HERE	The input is hierarchical or path-shaped; Subtree results combine recursively; Ordering can eliminate one branch; Keys share prefixes.
PREPARE FIRST	Volumes 1-12; Recursion, queues, and binary search.
MOVE ON WHEN	Implement and explain recursive and iterative traversals; Reason about depth, paths, LCA, and BST invariants; Serialize, reconstruct, and validate tree structure; Build and query a trie.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

12 - Binary Search	YOU ARE HERE 13 - Trees, BSTs, and Tries	14 - Heaps and Priority Queues
--------------------	--	--------------------------------

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Trees, Binary Search Trees, and Tries for SDE-2	4
Part II - Practice and Handoff	22
Practice Ladder and Next Steps	22

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - Trees, Binary Search Trees, and Tries for SDE-2

Tree questions combine recursive contracts with representation choices. A traversal is easy to memorize; the interview signal is whether you can say what a helper returns, where state lives, and why information from children is sufficient. Binary search trees add an ordering invariant that spans entire subtrees. Tries add prefix state and a potentially expensive branching representation. This chapter keeps heaps and ordered-map APIs out of scope so tree ownership and search invariants receive full attention.

Recognition and traversal map

Prompt signal	Traversal/state	Reason
parent before children, copy/serialize	preorder DFS	decision is made on entry
sorted values from a BST	inorder DFS	left, node, right follows ordering
children before parent, aggregate height	postorder DFS	parent depends on child summaries
levels, minimum unweighted depth	BFS	FIFO preserves distance layers
path sum / root-to-leaf condition	DFS with path state	state belongs to one current path
nearest shared ancestor	postorder returned evidence	combine whether targets occur below
ordered lookup/update	BST comparison	discard a full subtree per comparison
prefix lookup/autocomplete	trie walk by symbol	path itself is the prefix

Before choosing recursion, estimate height. A balanced tree with n nodes has height $O(\log n)$; a skewed tree has height n . Java does not guarantee tail-call elimination. An iterative traversal or explicit frame stack is the safe production choice for adversarial depth.

The structural and ownership model

A conventional tree is acyclic, has one root, and gives every non-root node exactly one parent. A Java `Node` object does not enforce those properties. Callers can create cycles or share a child between parents, turning the structure into a graph. Most interview algorithms assume a proper tree; state it.

Mutation contracts matter. BST insert/delete below reuse and rewire input nodes. Reconstruction allocates new nodes. Traversals do not mutate. A node exposed to multiple owners cannot be safely rewired without coordination. Production APIs often hide nodes, use immutable nodes, or confine mutations behind a tree abstraction.

Family 1: depth-first and breadth-first traversals

Recursive traversals

For preorder, the contract can be: "append the preorder sequence of `node`'s subtree and leave earlier output unchanged." The base case for `null` appends nothing. Append the node, then recursively append left and right. Inorder moves the append between children; postorder moves it after children.

Correctness follows by structural induction. An empty tree is correct. Assuming each child call returns its correct traversal, placing the root in the traversal-defined position yields the correct sequence for the parent. Every call moves to a child, so an acyclic finite tree terminates.

Time is $O(n)$ because every node is visited once. Call-stack space is $O(h)$ for height h ; returned output is $O(n)$ and is not auxiliary workspace.

Iterative inorder

Use a stack and cursor. The invariant is that the stack holds a path of nodes whose left subtrees have been scheduled or processed but whose own values have not yet been emitted; `current` heads a not-yet-descended subtree. Push the entire left spine. Pop one node, emit it, then explore its right subtree.

For tree 2 with children 1 and 3, push 2, 1; pop/emit 1; pop/emit 2; move to/push/pop 3. Output is [1, 2, 3]. Each node is pushed and popped once: $O(n)$ time, $O(h)$ space.

Level-order BFS

Enqueue the root. At the beginning of each outer iteration, queue size equals exactly the number of nodes in the next level. Remove that many, collect their values, and enqueue children. FIFO order ensures levels do not mix. Time $O(n)$; frontier space is $O(w)$, maximum width w , which can be $O(n)$ even when height is small.

Marking visited is unnecessary under the proper-tree contract. If shared/cyclic input is possible, use an identity-based visited set and redefine what traversal means.

Family 2: height, balance, diameter, and path sum

Height and balance

Define height before coding: this chapter uses number of nodes on the longest root-to-leaf path, so empty height is zero and leaf height is one. Height obeys $1 + \max(\text{leftHeight}, \text{rightHeight})$.

A naive balance check recomputes height at every node and can cost $O(n^2)$ on a skewed tree. Instead, use a postorder helper that returns subtree height when balanced and sentinel -1 when unbalanced. The contract lets failure propagate immediately. At a node, obtain both child results; reject if either is -1 or their height difference exceeds one; otherwise return one plus their maximum. Every node is processed once: $O(n)$ time, $O(h)$ stack.

Diameter

Define diameter as number of edges on the longest path between any two nodes. Postorder height in nodes gives a through-node edge count $\text{leftHeight} + \text{rightHeight}$. Update a shared accumulator, then return this node's height. The invariant is that after processing a subtree, the accumulator holds the greatest diameter found anywhere in it and the return value is exactly its height.

For a root with left chain of two nodes and right leaf, child heights are 2 and 1 ; through-root diameter is 3 edges. The longest path need not pass through the global root, so computing only root-left-height plus root-right-height is insufficient.

Path sum

For root-to-leaf target sum, subtract the current value from a **long** remainder. At a leaf, accept when remainder equals its value (or after subtracting, remainder is zero). A node with one null child is not a leaf. The helper explores each path independently, so no mutable global sum is needed. Time $O(n)$, stack $O(h)$.

Negative values invalidate pruning rules such as "stop when remainder becomes negative." Use that prune only when the value domain is nonnegative and explicitly guaranteed.

Family 3: lowest common ancestor

For a general binary tree where both target node identities are guaranteed present, use postorder:

- return the current node if it is **p** or **q**;
- recursively obtain evidence from left and right;
- if both are non-null, current is their lowest common ancestor;
- otherwise propagate whichever non-null node was found.

The helper returns null if neither target occurs, a target/ancestor if one side contains evidence, or the LCA once both sides contain evidence. Postorder ensures the first node combining two sides is lowest. Complexity is $O(n)$ time and $O(h)$ stack.

If either target may be absent, the basic result can wrongly return the one present node. Extend the return type with a found-count, or validate membership separately. Compare node identity, not equal values. In a

BST, value ordering can reduce LCA to $O(h)$ when keys are unique and both presence assumptions are defined.

Dry-run with root 6, targets 2 and 8: left returns 2, right returns 8, so root is LCA. For targets 2 and descendant 4, the call at node 2 returns itself immediately, and ancestors propagate it; 2 is correctly the LCA.

Family 4: reconstruction and serialization

Reconstruct from preorder and inorder

Preorder supplies the next subtree root. Inorder splits values belonging to its left and right subtrees. With unique values, map each inorder value to its index. The helper receives an inorder interval; consume one preorder value as root, verify its mapped index lies inside, recursively build left interval then right interval.

For preorder `[3,9,20,15,7]` and inorder `[9,3,15,20,7]`, root 3 splits inorder into `[9]` and `[15,20,7]`. The next preorder value builds left node 9. Next 20 splits the right interval into children 15 and 7.

Each node is mapped and consumed once: $O(n)$ time and $O(n)$ map plus $O(h)$ stack. Without a map, repeated inorder scanning can cost $O(n^2)$. Duplicate values make the traversals insufficient to determine a unique tree unless extra identity information or a deterministic convention is provided.

Serialization

Preorder with explicit null markers is self-delimiting: write value, then left, then right; write `#` for null. Deserialization consumes tokens under the same contract. For root 2 with children 1, 3, serialization is `2,1,#,#,3,#,#`. Null markers are essential: preorder values alone cannot distinguish many shapes.

Round-trip properties are stronger than one expected string: `deserialize(serialize(tree))` should have the same structure and values, and serializing the restored tree should produce the canonical string. Define escaping if values are not plain integers, version the format if persisted, cap depth/input, and treat deserialization as an untrusted boundary.

Family 5: binary search tree operations

Global invariant

This chapter's BST contract uses unique keys:

Every key in a node's left subtree is strictly less than the node key; every key in its right subtree is strictly greater.

Validation must enforce ancestor bounds, not only compare a node with immediate children. Tree 10 with left child 5 whose right child is 12 passes local parent-child checks but violates the root's left-subtree bound. Carry exclusive lower and upper `long` bounds; using `long` avoids overflow tricks around `Integer.MIN_VALUE` and `MAX_VALUE`.

Validation visits all nodes: $O(n)$ time, $O(h)$ stack. Inorder strict increase is another valid proof under the unique-key contract.

Search and insertion

At node x , if target is smaller, the global invariant proves the entire right subtree is impossible; go left. Greater goes right. Each comparison descends one level, so time is $O(h)$: $O(\log n)$ if balanced, $O(n)$ if skewed.

Insertion follows the same path until a null child is found. The path bounds prove attaching the new key there preserves all ancestor constraints. Define duplicate policy: reject, ignore, count frequency, or consistently choose a side. The sample ignores duplicate insertion.

Deletion

Deletion has three structural cases:

- 1 no left child: replace node with right child;
- 2 no right child: replace node with left child;
- 3 two children: choose inorder successor, the minimum of the right subtree; copy its key into the node, then delete that successor from the right subtree.

The successor is the smallest key still greater than every left-subtree key. It has no left child, so its recursive deletion reduces to an easier case. For deleting 5 from BST keys $[5, 3, 7, 6, 8]$, successor 6 replaces 5, and old 6 is removed below 7.

Time is $O(h)$, recursive stack $O(h)$. If nodes have identity observed by clients, copying only a key can violate object-level expectations. A production API may transplant nodes instead, store immutable entries, or hide node identity.

Kth smallest

BST inorder is sorted. Iterative inorder with a counter returns when the k th node is popped. Time is $O(h+k)$ in a balanced tree and $O(n)$ worst case; stack $O(h)$. Repeated rank queries justify augmenting every node with subtree size, but then rotations/inserts/deletes must maintain a new invariant.

Family 6: tries and prefixes

A trie stores one edge per symbol. The node reached after consuming prefix p represents exactly all inserted words beginning with p . Insert walks/creates edges, then marks a terminal node. Exact search requires the terminal mark; prefix search requires only that the path exists. A pass count can answer how many inserted words share a prefix.

For words **car**, **card**, **care**, and **dog**, path **c-a-r** has pass count three and terminal true for **car**. **ca** is a prefix but not an exact word. Search/insert costs $O(L)$ for word length L , independent of number of words, assuming constant-time edge access.

An array of 26 child references is fast for normalized lowercase English but wastes memory in sparse nodes and does not support arbitrary Unicode. A hash map is sparse but adds object/hash overhead and nondeterministic iteration unless ordered explicitly. Compressed radix trees, ternary search trees, finite-state structures, or database indexes may be better at scale. Always define normalization, case folding, locale, Unicode code point handling, maximum length, and duplicate semantics.

Complete Java 21 reference implementation

Compile and run with `java -ea TreesBstsTriesSde2`. The node algorithms assume a proper acyclic tree. The trie accepts only lowercase ASCII `a` through `z` so its representation contract is visible.

JAVA (continued 1/10)

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.NoSuchElementException;

public final class TreesBstsTriesSde2 {
    private TreesBstsTriesSde2() {}

    public static final class Node {
        public int value;
        public Node left;
        public Node right;
        public Node(int value) { this.value = value; }
    }

    public static List<Integer> preorderRecursive(Node root) {
        List<Integer> answer = new ArrayList<>();
        preorder(root, answer);
        return answer;
    }

    private static void preorder(Node node, List<Integer> answer) {
        if (node == null) return;
        answer.add(node.value);
        preorder(node.left, answer);
        preorder(node.right, answer);
    }

    public static List<Integer> inorderIterative(Node root) {
        List<Integer> answer = new ArrayList<>();
        Deque<Node> stack = new ArrayDeque<>();
```

JAVA (continued 2/10)

```
Node current = root;
while (current != null || !stack.isEmpty()) {
    while (current != null) {
        stack.push(current);
        current = current.left;
    }
    current = stack.pop();
    answer.add(current.value);
    current = current.right;
}
return answer;
}

public static List<List<Integer>> levelOrder(Node root) {
    if (root == null) return List.of();
    List<List<Integer>> answer = new ArrayList<>();
    ArrayDeque<Node> queue = new ArrayDeque<>();
    queue.addLast(root);
    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> level = new ArrayList<>(levelSize);
        for (int i = 0; i < levelSize; i++) {
            Node node = queue.removeFirst();
            level.add(node.value);
            if (node.left != null) queue.addLast(node.left);
            if (node.right != null) queue.addLast(node.right);
        }
        answer.add(level);
    }
    return answer;
}

public static int height(Node root) {
    return root == null ? 0 : 1 + Math.max(height(root.left), height(root.right));
}
```

JAVA (continued 3/10)

```
}

public static boolean isHeightBalanced(Node root) {
    return balancedHeight(root) >= 0;
}

private static int balancedHeight(Node node) {
    if (node == null) return 0;
    int left = balancedHeight(node.left);
    if (left < 0) return -1;
    int right = balancedHeight(node.right);
    if (right < 0 || Math.abs(left - right) > 1) return -1;
    return 1 + Math.max(left, right);
}

public static int diameterEdges(Node root) {
    int[] best = {0};
    diameterHeight(root, best);
    return best[0];
}

private static int diameterHeight(Node node, int[] best) {
    if (node == null) return 0;
    int left = diameterHeight(node.left, best);
    int right = diameterHeight(node.right, best);
    best[0] = Math.max(best[0], left + right);
    return 1 + Math.max(left, right);
}

public static boolean hasRootToLeafSum(Node root, long target) {
    if (root == null) return false;
    if (root.left == null && root.right == null) return target == root.value;
    long remaining = target - root.value;
    return hasRootToLeafSum(root.left, remaining)
        || hasRootToLeafSum(root.right, remaining);
}
```

JAVA (continued 4/10)

```

        || hasRootToLeafSum(root.right, remaining);
    }

    public static Node lowestCommonAncestor(Node root, Node p, Node q) {
        if (root == null || root == p || root == q) return root;
        Node left = lowestCommonAncestor(root.left, p, q);
        Node right = lowestCommonAncestor(root.right, p, q);
        if (left != null && right != null) return root;
        return left != null ? left : right;
    }

    public static Node buildFromPreorderInorder(int[] preorder, int[] inorder) {
        if (preorder == null || inorder == null || preorder.length != inorder.length) {
            throw new IllegalArgumentException("traversal lengths differ or are null");
        }
        Map<Integer, Integer> position = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) {
            if (position.put(inorder[i], i) != null) {
                throw new IllegalArgumentException("values must be unique");
            }
        }
        int[] preorderIndex = {0};
        Node root = build(preorder, 0, inorder.length, preorderIndex, position);
        if (preorderIndex[0] != preorder.length) {
            throw new IllegalArgumentException("inconsistent traversals");
        }
        return root;
    }

    private static Node build(int[] preorder, int inLow, int inHigh,
                              int[] preorderIndex, Map<Integer, Integer> position) {
        if (inLow == inHigh) return null;
        if (preorderIndex[0] >= preorder.length) {
            throw new IllegalArgumentException("inconsistent traversals");
        }

```

JAVA (continued 5/10)

```
    }
    int value = preorder[preorderIndex[0]++];
    Integer split = position.get(value);
    if (split == null || split < inLow || split >= inHigh) {
        throw new IllegalArgumentException("inconsistent traversals");
    }
    Node root = new Node(value);
    root.left = build(preorder, inLow, split, preorderIndex, position);
    root.right = build(preorder, split + 1, inHigh, preorderIndex, position);
    return root;
}

public static String serialize(Node root) {
    StringBuilder output = new StringBuilder();
    serialize(root, output);
    return output.toString();
}

private static void serialize(Node node, StringBuilder output) {
    if (node == null) {
        output.append("#,");
        return;
    }
    output.append(node.value).append(',');
    serialize(node.left, output);
    serialize(node.right, output);
}

public static Node deserialize(String encoded) {
    if (encoded == null || encoded.isEmpty()) {
        throw new IllegalArgumentException("empty encoding");
    }
    String[] tokens = encoded.split(",");
    int[] at = {0};
}
```


JAVA (continued 6/10)

```
Node root = deserialize(tokens, at);
if (at[0] != tokens.length) throw new IllegalArgumentException("extra tokens");
return root;
}

private static Node deserialize(String[] tokens, int[] at) {
    if (at[0] == tokens.length) throw new IllegalArgumentException("truncated tree");
    String token = tokens[at[0]++];
    if (token.equals("#")) return null;
    final int value;
    try { value = Integer.parseInt(token); }
    catch (NumberFormatException error) {
        throw new IllegalArgumentException("bad node value", error);
    }
    Node node = new Node(value);
    node.left = deserialize(tokens, at);
    node.right = deserialize(tokens, at);
    return node;
}

public static boolean validBst(Node root) {
    return validBst(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

private static boolean validBst(Node node, long lower, long upper) {
    if (node == null) return true;
    if (node.value <= lower || node.value >= upper) return false;
    return validBst(node.left, lower, node.value)
        && validBst(node.right, node.value, upper);
}

public static Node bstSearch(Node root, int target) {
    Node current = root;
```

JAVA (continued 7/10)

```
        while (current != null && current.value != target) {
            current = target < current.value ? current.left : current.right;
        }
        return current;
    }

    public static Node bstInsert(Node root, int value) {
        if (root == null) return new Node(value);
        Node current = root;
        while (true) {
            if (value == current.value) return root;
            if (value < current.value) {
                if (current.left == null) { current.left = new Node(value); return root; }
                current = current.left;
            } else {
                if (current.right == null) { current.right = new Node(value); return root; }
                current = current.right;
            }
        }
    }

    public static Node bstDelete(Node root, int value) {
        if (root == null) return null;
        if (value < root.value) root.left = bstDelete(root.left, value);
        else if (value > root.value) root.right = bstDelete(root.right, value);
        else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;
            Node successor = root.right;
            while (successor.left != null) successor = successor.left;
            root.value = successor.value;
            root.right = bstDelete(root.right, successor.value);
        }
    }
}
```

JAVA (continued 8/10)

```
        return root;
    }

    public static int kthSmallest(Node root, int k) {
        if (k <= 0) throw new IllegalArgumentException("k must be positive");
        Deque<Node> stack = new ArrayDeque<>();
        Node current = root;
        while (current != null || !stack.isEmpty()) {
            while (current != null) { stack.push(current); current = current.left; }
            current = stack.pop();
            if (--k == 0) return current.value;
            current = current.right;
        }
        throw new NoSuchElementException("k exceeds node count");
    }

    public static final class Trie {
        private static final class TrieNode {
            final TrieNode[] children = new TrieNode[26];
            int passCount;
            boolean terminal;
        }
        private final TrieNode root = new TrieNode();

        public void insert(String word) {
            requireWord(word);
            TrieNode node = root;
            node.passCount++;
            for (int i = 0; i < word.length(); i++) {
                int edge = edge(word.charAt(i));
                if (node.children[edge] == null) node.children[edge] = new TrieNode();
                node = node.children[edge];
                node.passCount++;
            }
        }
    }
}
```

JAVA (continued 9/10)

```
    }
    node.terminal = true;
}

public boolean contains(String word) {
    TrieNode node = find(word);
    return node != null && node.terminal;
}

public boolean hasPrefix(String prefix) {
    return find(prefix) != null;
}

public int wordsWithPrefix(String prefix) {
    TrieNode node = find(prefix);
    return node == null ? 0 : node.passCount;
}

private TrieNode find(String text) {
    requireWord(text);
    TrieNode node = root;
    for (int i = 0; i < text.length(); i++) {
        node = node.children[edge(text.charAt(i))];
        if (node == null) return null;
    }
    return node;
}

private static int edge(char ch) {
    if (ch < 'a' || ch > 'z') {
        throw new IllegalArgumentException("only lowercase a-z supported");
    }
    return ch - 'a';
}
```

JAVA (continued 10/10)

```

    }

    private static void requireWord(String text) {
        if (text == null) throw new IllegalArgumentException("text is null");
    }
}

public static void main(String[] args) {
    Node root = buildFromPreorderInorder(
        new int[] {3, 9, 20, 15, 7}, new int[] {9, 3, 15, 20, 7});
    assert preorderRecursive(root).equals(List.of(3, 9, 20, 15, 7));
    assert inorderIterative(root).equals(List.of(9, 3, 15, 20, 7));
    assert levelOrder(root).equals(List.of(List.of(3), List.of(9, 20), List.of(15, 7)));
    assert height(root) == 3 && isHeightBalanced(root);
    assert diameterEdges(root) == 3;
    assert hasRootToLeafSum(root, 30);
    assert lowestCommonAncestor(root, root.right.left, root.right.right) == root.right;
    String encoded = serialize(root);
    assert serialize(deserialize(encoded)).equals(encoded);

    Node bst = null;
    for (int value : new int[] {5, 3, 7, 2, 4, 6, 8}) bst = bstInsert(bst, value);
    assert validBst(bst) && bstSearch(bst, 6).value == 6;
    assert kthSmallest(bst, 3) == 4;
    bst = bstDelete(bst, 5);
    assert validBst(bst) && inorderIterative(bst).equals(List.of(2, 3, 4, 6, 7, 8));

    Trie trie = new Trie();
    trie.insert("car"); trie.insert("card"); trie.insert("care"); trie.insert("dog");
    assert trie.contains("car") && !trie.contains("ca");
    assert trie.hasPrefix("ca") && trie.wordsWithPrefix("car") == 3;
}
}

```

Complexity matrix

Operation	Time	Auxiliary space	Qualification
traversal / height / balance / diameter	$O(n)$	$O(h)$ or BFS $O(w)$	proper tree assumed
LCA in general tree	$O(n)$	$O(h)$	basic helper assumes both identities exist
reconstruct traversals	$O(n)$	$O(n)$ map + $O(h)$ stack	unique, consistent values
serialize/deserialize	$O(n)$	$O(h)$ stack + output	format and depth limits matter
BST search/insert/delete	$O(h)$	iterative $O(1)$ or recursive $O(h)$	h may be n unless balanced
kth smallest	$O(h+k)$ typical	$O(h)$	no subtree-size augmentation
trie operation	$O(L)$	insertion up to $O(L \cdot \text{alphabet slots})$	representation dominates memory

Avoid saying BST work is simply $O(\log n)$. That bound requires a balancing guarantee such as AVL or red-black invariants; the basic tree above can become a linked list.

WATCH OUT | Edge cases and common mistakes

- Define height and diameter in nodes or edges; off-by-one answers often come from switching definitions.
- A null root may mean empty output, zero height, or false path existence depending on contract.
- Recursive state stored in mutable fields can leak across calls; prefer returned summaries or method-local holders.
- BFS queue size must be captured before processing a level because children are added during the loop.
- Balance should compute child heights once, not rescan subtrees.
- LCA must clarify target presence and identity versus value.
- Reconstruction from preorder/inorder needs unique identity or a duplicate policy.
- Null markers preserve serialization shape; delimiters and malformed-input handling must be explicit.
- BST validation requires ancestor bounds, not only parent-child comparisons.
- Duplicate BST keys require a consistent global policy.
- BST delete can change the root; callers must use the returned root.
- A trie prefix is not necessarily a complete stored word.
- Fixed child arrays trade memory for speed and only fit a declared alphabet.

TRY IT | Exercises with model checkpoints

Exercise 1: iterative preorder and postorder

Implement both without recursion.

Checkpoint: preorder can push right then left so left is processed first. Postorder may use two stacks or one stack with `(node, visited)` frames. State the frame invariant and $O(h)$ versus potentially $O(n)$ explicit storage.

Exercise 2: right-side view

Return the value visible from the right at every depth.

Checkpoint: BFS may record the final node of each captured level, or DFS may visit right before left and record the first value seen at a new depth. Both are $O(n)$; DFS stack risks skewed depth while BFS pays width.

Exercise 3: LCA with missing targets

Return an answer only if both node identities occur.

Checkpoint: return a record containing candidate and a two-bit/found count. Combine child counts plus current identity, and publish an LCA only at count two. Do not run two full membership traversals unless simplicity is preferred over one pass.

Exercise 4: iterative safe deserialization

Handle a persisted tree whose depth may be hostile.

Checkpoint: use explicit frames describing the next child slot, cap nodes/tokens/depth, reject extra/truncated tokens, and avoid recursive stack exhaustion. Include a version and integrity/authentication decision for untrusted storage.

Exercise 5: balanced BST guarantee

Explain how an AVL or red-black tree changes complexity.

Checkpoint: it maintains additional rotation/color/height invariants so height remains $O(\log n)$. Operations pay rebalancing work but preserve worst-case lookup/update. Do not attempt to reimplement a production balanced map in an interview unless asked; focus on invariants.

Exercise 6: trie deletion

Delete one word while retaining shared prefixes.

Checkpoint: clear terminal only if present, decrement pass counts along the path, and remove a child link only when its pass count becomes zero. Define duplicate insertion: a boolean terminal cannot distinguish inserting the same word twice; use terminal frequency if multiset behavior is required.

Exercise 7: autocomplete top-k

Return the most frequent completions under a prefix.

Checkpoint: a naive subtree traversal is output/subtree dependent. Storing top-k summaries at prefix nodes speeds reads but increases update cost and consistency obligations. Define ranking ties, freshness, normalization, memory budget, and whether results are eventually consistent.

SDE-2 production follow-ups

How do you protect against skew? Use a balanced tree, iterative algorithms, depth limits, or a different index. Random input is not a guarantee. Monitor height or operation latency if the structure evolves from external keys.

Would you serialize raw object graphs? Prefer a versioned schema with validation, size/depth limits, and explicit fields. Never assume deserialized data is a proper tree. Avoid Java native serialization for untrusted boundaries; construction should enforce invariants before publication.

How does concurrency affect tree mutation? Insert/delete rewires several references while preserving a global order. Unsynchronized readers can see partial structure. Use confinement, immutable/persistent trees, copy-on-write snapshots for read-heavy workloads, or a proven concurrent index. **volatile** child fields alone do not make rotations or deletion atomic.

How do you choose trie representation? Measure alphabet size, sparsity, word count, prefix distribution, latency, and update frequency. Arrays optimize predictable small alphabets; maps optimize sparse edges; compressed tries reduce unary chains. Normalize text consistently at ingestion and query.

What tests establish confidence? For traversals, compare recursive and iterative outputs. For serialization, use round-trip properties. For BSTs, after random operations verify strict inorder order and a reference set. For tries, compare prefix counts against a simple list oracle. Include empty, singleton, skewed, duplicate, minimum/maximum key, malformed encoding, and unsupported-symbol inputs.

Final readiness checklist

- I choose preorder, inorder, postorder, or BFS from information flow.
- Every helper has a return contract and an explicit empty-tree identity.
- I count height h and frontier width w , not only node count.
- My LCA answer states presence and identity assumptions.
- Reconstruction and serialization preserve shape, not merely values.
- BST validation carries ancestor bounds and declares duplicate policy.
- I say $O(h)$, then explain when height is logarithmic or linear.
- Trie alphabet, normalization, terminal state, prefix counts, and memory trade-offs are explicit.

Tree mastery is disciplined information flow: children return exactly what parents need, ordering lets a branch be eliminated only when a global invariant permits it, and representation choices remain visible in both correctness and production cost.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: traversals, height, and search
- Interview Core: validate BST, LCA, and path problems
- SDE-2 Follow-up: iterative traversal and state ownership
- Challenge: serialization or prefix-search design

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous volume: 12 - Binary Search: Bounds, Answers, and Invariants](#)

[Series index](#)

[Next volume: 14 - Heaps, Priority Queues, Selection, and Top-K](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 13 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.